

UNIVERSIDAD DE CARABOBO
FACULTAD DE CIENCIAS Y TECNOLOGÍA
DEPARTAMENTO DE COMPUTACIÓN

PRÁCTICA DE LABORATORIO N° 1

Familiarización con MIPS32 y el Entorno MARS

Autores:

Genesis V. Cabello M.

C.I:30.367.386

Maivet S. Pereira M.

C.I: 30.814.708

Profesor:

Jose Canache

Asignatura:

Arquitectura del Computador

1. Objetivo

Familiarizar al estudiante con la sintaxis del lenguaje ensamblador MIPS32 y su ejecución en el entorno MARS, resolviendo problemas clásicos de programación, prestando especial atención a la gestión de registros, llamadas a funciones, recursividad, estructuras de control, manejo de la pila y configuración/utilización del ambiente de trabajo.

2. Investigación Preliminar

2.1. ¿Cómo se implementa la recursividad en MIPS32? ¿Qué papel cumple la pila (*sp*)?

Implementación de la recursividad: En MIPS32, la recursividad se logra mediante la instrucción jal(Jump and Link), la cual realiza un salto hacia la etiqueta de la misma función, guardando automáticamente la dirección de la siguiente instrucción en el registro \$ra(Return Address).

El papel de la pila (\$sp): Dado que cada llamada con jal sobrescribe el registro \$ra, si no se salvara este valor, el programa perdería la referencia de cómo regresar a la función que la llamada, entrando en un bucle infinito. El puntero de pila (\$sp) nos permite reservar memoria en el Stack para almacenar (o "proteger") el registro \$ra y los argumentos locales (\$a0) de cada nivel de la recursión antes de hacer la nueva llamada. Al regresar, se restauran estos valores usando lw y se libera el espacio.

2.2. ¿Qué riesgos de desbordamiento existen? ¿Cómo mitigarlos?

El riesgo (Stack Overflow): Cada llamada recursiva consume una cantidad fija de memoria en la pila (en nuestro código, 8 bytes). Si el usuario ingresa un número n extremadamente grande, la pila crecerá tanto que superará los límites físicos de la memoria asignada por el sistema operativo o el simulador, provocando un desbordamiento de pila (Stack Overflow) y colgando el programa.

Mitigación:

1.Validación de entrada: Limitar el valor máximo de n mediante software antes de llamar a la función.

2.Cambio de paradigma: Si el algoritmo requiere procesar volúmenes de datos masivos, la mejor mitigación es transformar el código recursivo en uno iterativo , ya que este último no consume memoria de la pila.

2.3. ¿Qué diferencias encontraste entre una implementación iterativa y una recursiva en cuanto al uso de memoria y registros?

Característica	Enfoque Recursivo	Enfoque Iterativo
Uso de la Pila (\$ap)	Alto (Reserva espacio por llamada)	Nulo (No modificado el \$sp)
Consumo de Memoria	Complejidad espacial $O(n)$	Complejidad espacial $O(1)$
Riesgo de <i>Stack overflow</i>	Sí (Con valores de n muy altos)	No existe riesgo
Uso de Registros criticos	Depende de \$ra y \$a0	Usa registros temporales (\$t0, \$t1)
Eficiencia Temporal	Más lento (Accesos frecuentes a RAM)	Más rápido (Operaciones directas en CPU)

2.4. ¿Qué diferencias encontraste entre los ejemplos académicos del libro y un ejercicio completo y operativo en MIPS32?

Los ejemplos académicos de los libros suelen mostrar únicamente el fragmento aislado de la función (la lógica pura), omitiendo las directivas de asignación de memoria o el manejo de entradas y salidas. Un ejercicio completo y operativo (como los que ejecutamos en MARS) requiere una estructura formal con directivas de datos (`.data`), código (`.text`), la declaración global del punto de entrada (`.globl main`), la gestión de llamadas al sistema (`Syscalls`) para interactuar con la consola, y una finalización limpia del programa (`Syscall 10`) para evitar que el procesador intente seguir ejecutando líneas de código vacías en la memoria.

2.5. Tutorial de la ejecución paso a paso en MARS

Para evaluar el comportamiento interno del procesador MIPS32 y el flujo de los algoritmos desarrollados, se realizó una depuración paso a paso (*Single-Step*) utilizando el simulador MARS. A continuación, se detallan las fases clave del ciclo de ejecución:

1. **Fase de Ensamblado e Inicialización:** Al cargar el archivo de código fuente (`.asm`) y presionar el ícono de la herramienta *Assemble*, el entorno traduce las instrucciones a lenguaje máquina y activa la pestaña *Execute*. El contador de programa (PC) se inicializa en la dirección base del `main` (`0x00400000`) y los registros se encuentran en su estado por defecto.

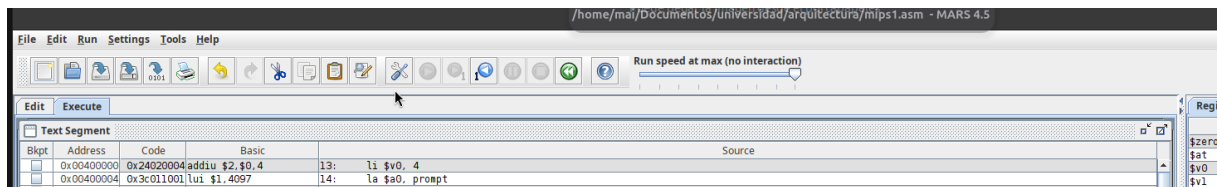


Figura 1: fase de inicializacion

2. **Fase de Entrada de Datos (Syscall):** Utilizando la tecla F7 para avanzar instrucción por instrucción, el procesador ejecuta las llamadas al sistema (*Syscalls*) para imprimir el mensaje de solicitud en la consola *Run I/O* y detenerse a esperar el ingreso del número por parte del usuario. Para este tutorial, se introdujo el valor $n = 3$, el cual se almacena temporalmente en el registro $\$v0$ y luego se transfiere a un registro seguro.

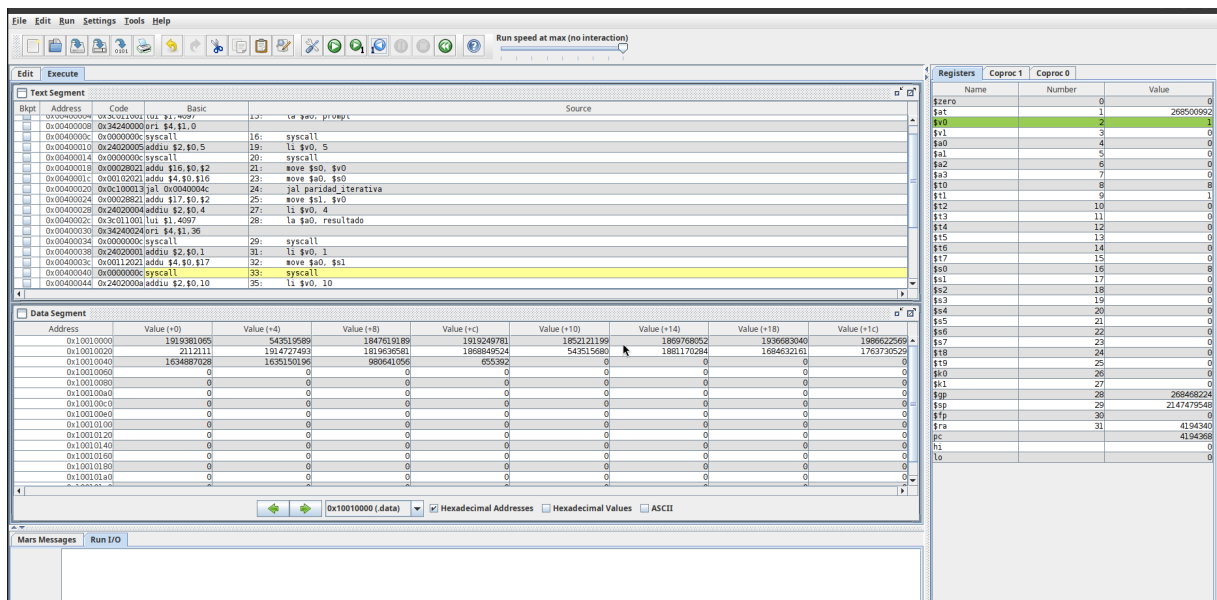


Figura 2: Fase 2 entrada de datos

3. **Fase Crítica: Comportamiento de la Pila (\$sp) en la Recursión:** Al invocar la función *paridad_recursiva* mediante la instrucción *jal*, se observa el núcleo de la práctica. Como $n = 3$ no cumple el caso base, el programa entra en la sección recursiva:

- El puntero de pila ($\$sp$) decrementa su valor en 8 bytes en cada llamada (*addi \$sp, \$sp, -8*), abriendo espacio en la memoria RAM. Ocurrido el decremento, se ejecutan las instrucciones *sw* para respaldar la dirección de retorno ($\$ra$) y el argumento actual ($\$a0$).

Este ciclo se repite sucesivamente decrementando n hasta que alcanza el valor de cero ($n = 0$), momento en el cual la pila alcanza su tamaño máximo de ocupación.

4. **Fase de Desenrollado y Retorno:** Al activarse el caso base, el registro $\$v0$ carga el valor de retorno inicial (0). A partir de aquí, cada pulsación de F7 ejecuta instrucciones *lw* que recuperan los contextos previos de la pila y aumentan el $\$sp$

de vuelta a su posición original (`addi $sp, $sp, 8`). Simultáneamente, se calcula la alternancia de paridad mediante la resta `sub $v0, $t0, $v0`, ascendiendo por los niveles de la llamada hasta retornar al `main` con el resultado final de 1.

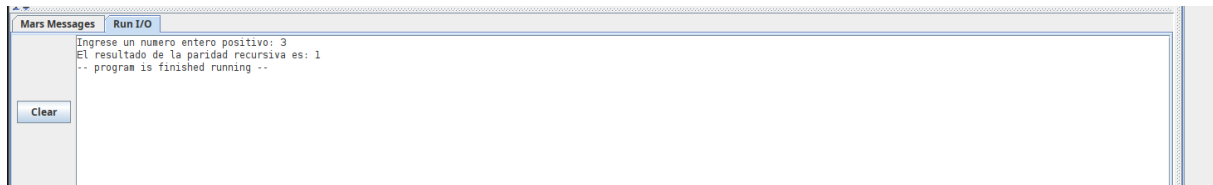


Figura 3: Enter Caption

2.6. Justificar la elección del enfoque (iterativo o recursivo) según eficiencia y claridad en MIPS

Para este problema en particular (calcular la paridad), el enfoque iterativo es infinitamente superior. **Eficiencia:** El enfoque iterativo es mucho más rápido y óptimo. El recursivo gasta ciclos de reloj del procesador ejecutando instrucciones de acceso a memoria (`sw` y `lw`) por cada decremento de n . En bajo nivel, acceder a la memoria RAM siempre es más lento que operar directamente con los registros del procesador. **Claridad:** Aunque a nivel matemático la definición recursiva es elegante, trasladarla a ensamblador MIPS32 se vuelve complejo y propenso a errores debido al manejo manual de la pila. El enfoque iterativo se traduce de forma limpia mediante un bucle convencional (`beqy j`), haciéndolo más legible y seguro de mantener.